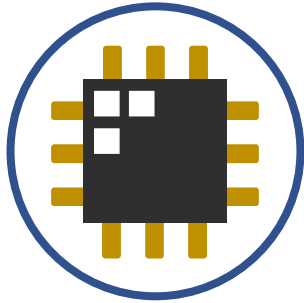
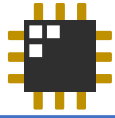




EEE 143: Switching Theory and Digital Logic Design

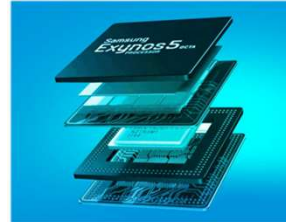
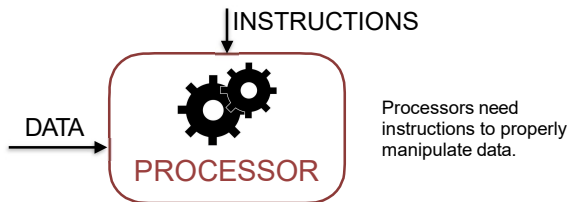
Lecture 2 – Logic gates, Truth Tables,
Boolean expressions

1



Recap: A computer system

Computers are *digital systems*

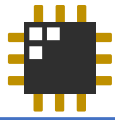


Digital systems manipulate digitized data using logic

- The basic building block of a digital system is a switch
 - Two states: **OFF** or **ON**... **LOW** or **HIGH**... **0** or **1** (binary digits)

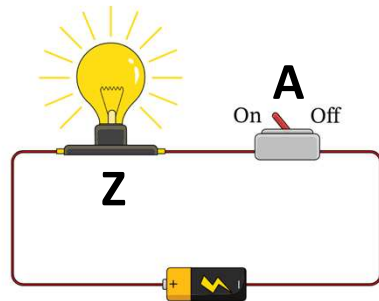
2

Quick recap, digital systems are all around us. One excellent example is the processor running a computer system. These are digital systems that are actually built out of networks of logic gates, or switches. For this lecture, we will now start covering what logic is and how we can implement useful systems using it.



Introduction to logic

- **Switches:** Basic element of a digital circuit
- If A is 1, the switch is closed, Z (light bulb) turns on
- If A is 0, the switch is open, Z (light bulb) turns off



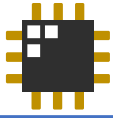
$$Z = A$$

Whatever the state of A is,
Z copies it

3

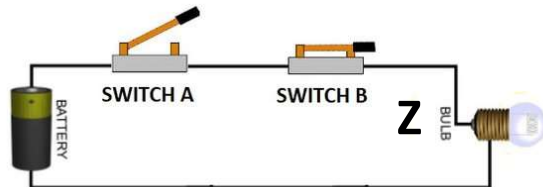
Now, let's consider this simple circuit with a switch and a light bulb. The switch can be in one of two states: on (1) or off (0).

The light bulb will light up when the switch is on (1) and will be off when the switch is off (0). In this case, the output, which is the light bulb represented as Z, exactly depends on the input, which is the switch represented as A. Thus, we can say that $Z = A$ in this example. That is an expression that we can use to describe this setup.



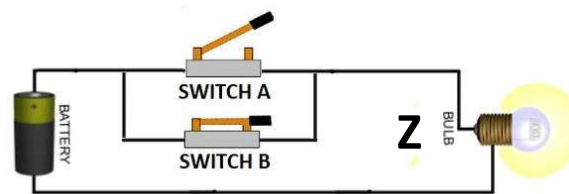
Introduction to logic

- We can make more complex ones!



$$Z = A \text{ AND } B$$

A and B needs to be 1 so that Z turns on (1)



$$Z = A \text{ OR } B$$

A or B needs to be 1 so that Z turns on (1)

Image credit: "How were computers made using very simple logic gates..." quora.com

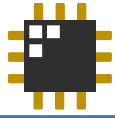
4

The previous circuit is not really interesting since the output just copies the input. However, we can include more switches and play around with the configuration for a bit.

Consider the first circuit at the top. Imagine you have two light switches in a row (series). Both switches need to be in the 'ON' position (1) for the light bulb at the end of this series circuit to light up. If either of the switches is in the 'OFF' position (0), even just one, the light bulb will not light up. We can write an expression describing this scenario. The light bulb is still the output Z and now we have two inputs. For the light bulb, Z, to turn on, both A AND B must be on. We have that AND condition there. That's one of the basic logic component being used in a computer system!

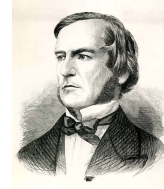
Now consider the second circuit at the bottom. Imagine you have two light switches in parallel. In this case, the light bulb will light up if either one of the switches or both are in the 'ON' position (1). The light bulb will only be off if both switches are in the 'OFF' position (0). So now, what is the expression for this one? For the light bulb, Z, to turn on, either A OR B must be on. We have that OR condition there. That is another basic logic component being used in

the world of computing.



Introduction to logic

- Early computer designers built ad hoc circuits from switches
 - They began to notice common 'patterns'/logic in their work: ANDs, ORs
- **George Boole** came up with a linguistic algebra to encode logic into a language that can be manipulated and solved mathematically
 - We call it *Boolean logic* in his honor.
- **Claude Shannon's** 1938 thesis linked switches and logic.
 - He is the first to describe *Boolean logic gates*



George Boole
(1815-1864)



Claude Shannon
(1916-2001)

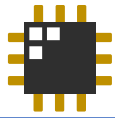
5

Now, before we proceed, a little bit of history just for added appreciation. Why are we presenting these switch analogy and AND and OR logic? The development of early computers involved building ad hoc circuits from switches. As these early computer designers worked, they began to notice common patterns or logic in their work, such as ANDs and ORs. These patterns are fundamental to how we process and interpret information.

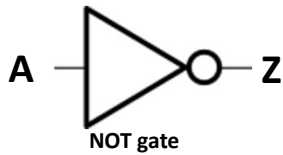
George Boole was the first to develop a linguistic algebra to encode logic into a language that could be manipulated and solved mathematically. This became known as Boolean logic, which was named in his honor. Boolean logic is the backbone of digital electronics and computing, allowing us to represent complex logical statements in a form that machines can understand and process.

Then, in his groundbreaking 1938 thesis, Claude Shannon made the crucial link between these physical switches and abstract logic, describing the first logic gates. This was a revolutionary idea that laid the foundation for digital circuit design.

These developments were pivotal in the evolution of computing, transforming it from a niche scientific tool into a ubiquitous part of modern life. The principles of AND, OR, and Boolean logic that were recognized and formalized by these pioneers are still fundamental to how our digital devices operate today.



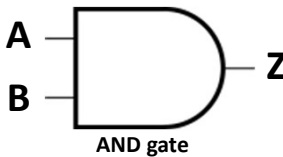
Introduction to logic



NOT gate

$$Z = A'$$

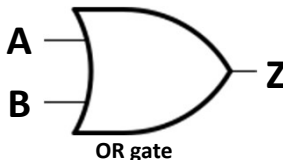
We usually say "A prime", or "not A"
Note: Some references would also denote this as
 $Z = \bar{A}$ (using overbar) or $Z = \sim A$ (using tilde)



AND gate

$$Z = AB$$

We usually say "A and B"
Note: Some references would also denote this as
 $Z = A \times B$ or $Z = A \cdot B$



OR gate

$$Z = A + B$$

We usually say "A or B"
Don't be fooled, this is not addition!

A	Z
0	1
1	0

A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

These are what we call
truth tables

Truth tables describe the
output given the
combination of inputs.

Hence, we typically refer
a **network of logic gates** as
combinational logic

Image credit: wikimedia.com

6

Now, let's go back. Here we have the three basic logic gates in digital circuits. The last two is something that you're already familiar about, AND and OR. The first one is what we call the NOT logic, which is basically just the inverse of whatever the input is.

On the left, we see the logic gate diagrams of the three. For the NOT gate, also called an inverter, it is just a triangle with a circle at the end. The circle at the end is the most important thing here, since that implies the negation, or the inversion of the input. AND gate looks like the one in the middle and the OR gate is the one below. Remember how their diagrams are drawn as they will show up frequently, especially as we move towards more complex circuits.

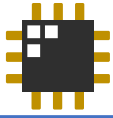
In the middle column, we have the equivalent logic statements that describes their functionality, together with the symbols we use to represent them. For a NOT gate, the output Z is just A prime. The prime there implies the inversion of whatever A is. The truth table, which describes the functionality of the logic is shown on the right. As you can see, Z is just the inverted version of A.

For the AND gate, it is the output Z being equal to A and B. We usually just write them together like this, as if multiplying them in math. Actually, if you think about it,

the AND logic is indeed similar to a 1-bit multiplication, don't you think? Looking at the truth table, the output Z only becomes 1 if both A and B are 1. That's also similar to a 1-bit binary multiplication, indeed.

For the OR gate, it is the output Z being equal to A or B. We usually write the OR operation like this, which is, yes, can be misleading. This is not an addition when we are discussing logic gates. This operator is the OR operator. Looking at the truth table for the OR gate, as described previously, the output Z becomes 1 if either A or B becomes 1.

We typically use truth tables to describe the functionality of a circuit. For more complex logic, it also allows us to derive what the equivalent logical expression should be.



Boolean Algebra and Set Theory

- In conventional algebra, where variables represent quantities, '+' and '×' are used to indicate addition and multiplication, respectively
- In **Boolean algebra**, variables represent truth values
 - + means OR operation, or the union of two sets
 - × means AND operation, or the intersection of two sets
 - We usually do not explicitly put the × symbol anymore and just stick variables together (just like in algebra)
 - ' indicates a NOT operation, or the inverse of a set
- Universal set is represented by '1'
- '0' or False means an empty set

7

In conventional algebra, we work with numbers, and the symbols '+' and '×' represent addition and multiplication, respectively. However, in Boolean algebra, variables don't represent numbers but rather truth values, it's either True (1) or False (0).

'+' (OR operation): Instead of addition, '+' represents the logical OR, which can be thought of as the union of two sets. If at least one value is true, the result is true.

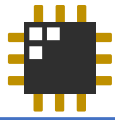
'×' (AND operation): Instead of multiplication, '×' represents the logical AND, which corresponds to the intersection of two sets. It's only true if both values are true.

'"' (NOT operation): The prime symbol (') represents the logical NOT, which is the inverse of a value. If something is true (1), its NOT is false (0), and vice versa.

Additionally, we have:

The universal set is represented by '1', meaning it contains everything (always true).

'0' or (False) represents an empty set, meaning there's nothing in it (always false).



Boolean Algebra and Set Theory

- Boolean logic is actually just set theory!

- Can you identify the logic being implied in the following diagrams?
(shaded means true/1)

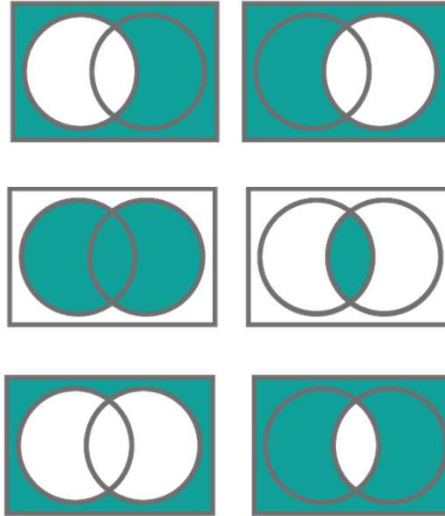
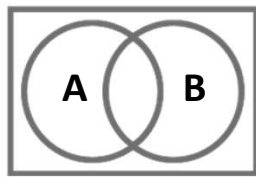


Image credit: "Bit arithmetic and Venn Diagrams" steemit.com

8

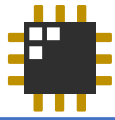
Another perspective in Boolean logic is set theory. The NOT, AND, and OR logic can be imagined in the way of Venn diagrams as well.

Look at these diagrams, assuming A is the left circle and B is the right circle inside the box, would you be able to identify the corresponding logic shown in the diagram?

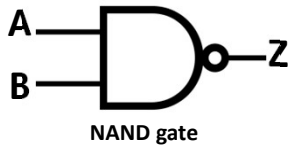
(top left) A' (top right) B'

(mid left) $A+B$ (mid right) AB

(bottom left) $(A+B)'$ (bottom right) $(AB)'$



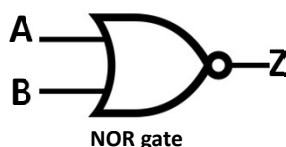
More logic gates



NAND gate

$$Z = (AB)'$$

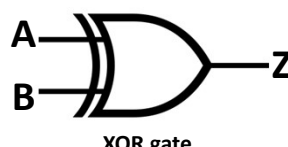
A	B	Z
0	0	1
0	1	1
1	0	1
1	1	0



NOR gate

$$Z = (A + B)'$$

A	B	Z
0	0	1
0	1	0
1	0	0
1	1	0

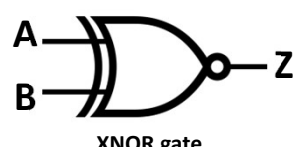


XOR gate

$$Z = A \oplus B$$

$$Z = A'B + AB'$$

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	0



XNOR gate

$$Z = (A \oplus B)'$$

$$Z = AB + A'B'$$

A	B	Z
0	0	1
0	1	0
1	0	0
1	1	1

These logic gates can also be implemented using AND, OR, NOT!

Image credit: wikimedia.com

9

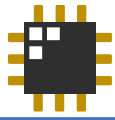
So far, we've discussed the three fundamental logic gates: **AND**, **OR**, and **NOT**. Now, let's explore **four additional gates** that build upon these basics:

NAND (NOT AND) Gate is simply the negation of AND gate. It outputs 0 only when both inputs are 1; otherwise, it outputs 1.

NOR (NOT OR) Gate is the negation of OR gate. It outputs 1 only if both inputs are 0; otherwise, it outputs 0.

XOR Gate outputs 1 if the inputs are different (one is 1 and the other is 0) and 0 if the inputs are the same.

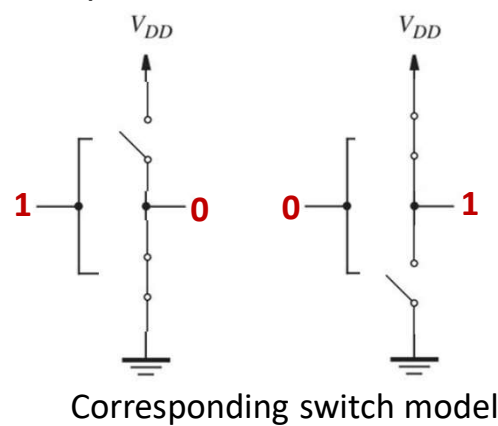
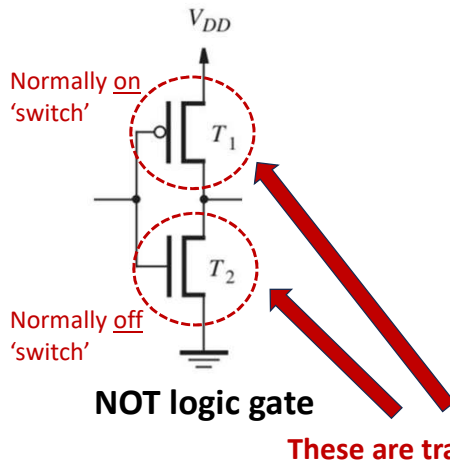
XNOR Gate is the complement of XOR. It outputs 1 if the inputs are the same and 0 if they are different.



Segue: Logic gates in real-life

Logic gates are just networks of switches.

Transistors are electrical components used as switches.



10

At their core, logic gates are just networks of switches that control the flow of electrical signals. These switches are implemented using transistors, which are electrical components that act as electronic switches.

Let's consider the example, a NOT gate built using two transistors: one normally ON and one normally OFF.

Understanding transistors as switches:

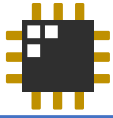
Normally on – conducts when the input is low (0) and turns off when the input is high (1).

Normally off – conducts when the input is high (1) and turns off when the input is low (0).

Looking on the corresponding switch model on the right:

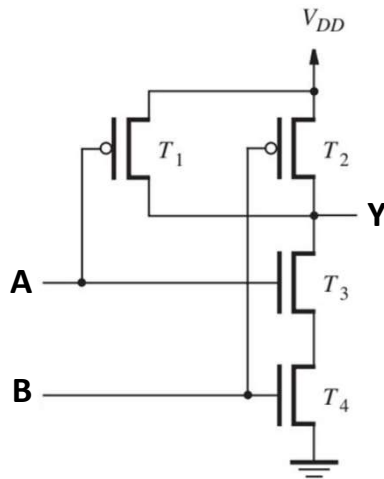
For an input = 1, the normally ON switch turns OFF (stop conducting) and the normally OFF switch turns ON, allowing current to flow to ground (0V), hence the output is low (0).

For an input = 0, the normally ON switch turns ON, connecting the output to V_{DD} (1) and the normally OFF switch turns OFF, blocking the path to ground, hence, the output is high (1).



Segue: Logic gates in real-life

NAND (NOT AND) logic gate



A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

**More complex logic:
more transistors!**

11

Again, here's the behavior of the two type of transistors:

Normally on – conducts when the input is low (0) and turns off when the input is high (1).

Normally off – conducts when the input is high (1) and turns off when the input is low (0).

So the circuit consists of two normally on transistors connected in parallel (at the top, pulling output HIGH) and two normally off transistors connected in series (at the bottom, pulling the output LOW).

Case 1: Both inputs is 0

Normally on switches are ON, connected the output to V_{DD} (1) and normally off switches are OFF, preventing the output from being pulled to ground, resulting for the output to be HIGH (1).

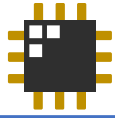
Case 2: One input is 0 and the other is 1

At least one normally on switch is ON, so the output remains connected to V_{DD} (1) and at least one normally off switch is OFF, preventing the output from being pulled to the ground, resulting for the output to be HIGH (1).

Case 3: Both inputs are 1

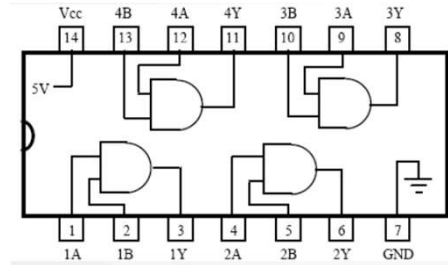
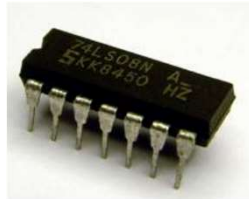
The normally on switches turn OFF, disconnecting the output from V_{DD} and the normally off switch turn ON, forming a direct path to ground (0), resulting for the output to be LOW (0).

This configuration ensures that the output is only LOW when both inputs are HIGH, which is the exact behavior of a NAND gate.



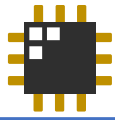
Segue: Logic gates in real-life

- Logic gates can come in an Integrated Circuit (IC) package
 - Example: 7408 Quad 2-Input AND Gate



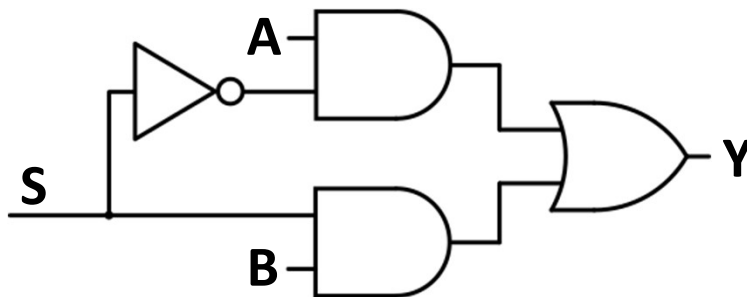
12

Logic gates are often grouped into Integrated Circuits (ICs), which are compact chips containing multiple gates in a single package. These ICs make it easier to design and implement digital circuits without needing individual transistor-based logic gates.



More complex logic

- We can make combinations of logic gates to make more complex logic!



S	A	B	Y
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

13

From what we've covered earlier, we figured that we can actually chain logic to create more complex ones.

Here's a rather complicated digital logic circuit, but everything you see are just the logic gates that you've seen before.

We have three inputs now, A, B and S. Since we have three inputs, each input can be either 0 or 1 independently of one another. Therefore, we have $2^3 = 8$ different possible combinations. The truth table here writes all the different combinations of the three inputs. As you can see, it is actually written in the order of increasing binary value. That's one way to guarantee that you have covered all the different combinations.

Now, let's try to write the equivalent Boolean expression of this logic circuit:

$$Y = S'A + SB$$

Then, let's complete the truth table by writing out the values of the output Y for every combination of S, A and B. You can approach this in two ways: either test all the possible input combinations in the circuit and see what the output should be for every combination; or look at the equivalent expression and see if you can complete the table from there.

Y
0
0
1
1
0
1
0
1

$$Y = S'A + SB$$

We can only look for the cases where Y becomes 1.

1.If S = 1:

1. Then S' (the inverted version of S) is 0.
2. So, S'A is 0 ANDed with A.
3. Looking at the AND logic, both inputs must be 1 for the output to be 1.
4. So, if one of the inputs is 0 (which is the case here), then the output will be 0.
5. Therefore, S'A will be 0 for all values of A.
6. However, since S is 1, SB will be B for all values of B.
7. So, $Y = S'A + SB$ will be 1 when B is 1.

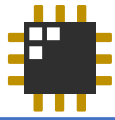
2.If S = 0:

1. Then S' is 1.
2. So, S'A is 1 ANDed with A.
3. Again, looking at the AND logic, both inputs must be 1 for the output to be 1.
4. So, S'A will be 1 when A is 1.
5. However, since S is 0, SB will be 0 for all values of B.
6. So, $Y = S'A + SB$ will be 1 when A is 1.

So, in summary, Y will be 1 in two cases:

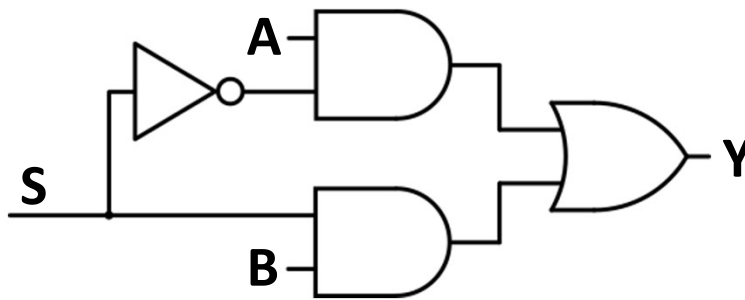
- When S is 1 and B is 1.
- When S is 0 and A is 1.

This is just a teaser! We will discuss more Boolean logic theory on our next meeting.



More complex logic

- We can make combinations of logic gates to make more complex logic!



$$Y = S'A + SB$$

S	A	B	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

14

Following the explanations from the previous slide, we have this Boolean expression and the corresponding completed truth table.

What we just did is formally called "analysis of combinational circuits -

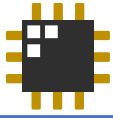
Given a logic diagram or logic circuit, determine the Boolean expression:

1. Analyze the circuit one gate at a time, systematically

going from the input side to the output side of the circuit

2. Replace each gate with its associated logical operator

3. Operands are the inputs to the gates



Truth tables to Boolean expression

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

- Boolean expressions and truth tables **represent the same thing**

- You can derive one from the other and vice versa!

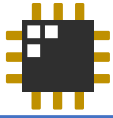
- Write the equivalent Boolean expression for the truth table on the left.

- **Tip:** Write the conditions that needs to be satisfied to make the output 1

Boolean expressions and truth tables are two different ways of representing the same logical relationships. They are interconnected and you can derive one from the other.

To derive a Boolean expression from a truth table, you can follow these steps:

1. Identify the rows in the truth table where the output is 1.



Truth tables to Boolean expression

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

_____?

_____?

_____?

_____?

- For the **first row**, we say **Q** becomes 1 if A is 0 **AND** B is 0 **AND** C is 0

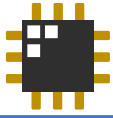
- If something *needs to be 0* to satisfy a condition, that implies the **NOT** logic

- If a *specific combination of inputs* need to be satisfied, that implies the **AND** logic

- Looking at a single row

2.) For each of these rows, write down a term that consists of each input variable. If the variable is 1 in this row, write it down as it is. If it's 0, write down its complement (i.e., with a NOT operation).

3.) Looking at a single row in the truth table, a specific combination of inputs that needs to be satisfied implies the logical AND. For example, in this first row. $Q = 1$ if and only if, A is 0, and B is 0, and C is 0. So all of them must be 0 to make the output Q to be 1.



Truth tables to Boolean expression

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

$A'B'C'$

_____?

- For the **first row**, we say Q becomes 1 if A is 0 AND B is 0 AND C is 0

A' AND B' AND C'

$A'B'C'$

- If something *needs to be 0* to satisfy a condition, that implies the **NOT** logic

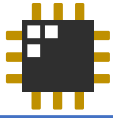
_____?

- If a *specific combination of inputs* need to be satisfied, that implies the **AND** logic

- Looking at a single row

_____?

So if you write that in Boolean expression form, it is literally the English statement earlier. $Q=1$, if A is 0 AND B is 0 AND C is 0. And there's your expression: A' AND B' AND C' . This is for the first row.

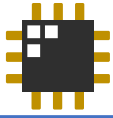


Truth tables to Boolean expression

A	B	C	Q	
0	0	0	1	$A'B'C'$
0	0	1	1	_____?
0	1	0	0	
0	1	1	0	
1	0	0	0	
1	0	1	1	_____?
1	1	0	0	
1	1	1	1	_____?

**Complete the
Boolean expressions
per row!**

So just do that for all the rows that has the output $Q = 1$.



Truth tables to Boolean expression

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

$A'B'C'$

$A'B'C$

$AB'C$

ABC

- If either of the several combinations of inputs can be satisfied to make the output 1, that implies the **OR** logic
 - Looking at multiple rows

Q becomes 1 if

row1 is 1 OR

row2 is 1 OR

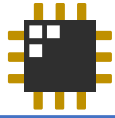
row6 is 1 OR

row8 is 1

Now, after getting all of those terms, we combine them by ORing the individual terms together.

So now, in English, this basically says that Q will be 1 if and only if, A is 0 and B is 0, and C is 0, OR A = 0 B = 0 C = 1, OR A = 1 B=0 C= 1, and so on. That's exactly the English translation. And thus, this expression exactly describes this truth table.

This is already the Boolean expression that is logically equivalent to the truth table presented in this slide.



Truth tables to Boolean expression

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

$A'B'C'$

$A'B'C$

$AB'C$

ABC

- If either of the several combinations of inputs can be satisfied to make the output 1, that implies the **OR** logic
 - Looking at multiple rows

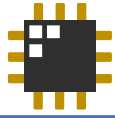
Q becomes 1 if

row1 is 1 OR	$A'B'C'$ +
row2 is 1 OR	$A'B'C$ +
row6 is 1 OR	$AB'C$ +
row8 is 1	ABC

Now, after getting all of those terms, we combine them by ORing the individual terms together.

So now, in English, this basically says that Q will be 1 if and only if, A is 0 and B is 0, and C is 0, OR A = 0 B = 0 C = 1, OR A = 1 B=0 C= 1, and so on. That's exactly the English translation. And thus, this expression exactly describes this truth table.

This is already the Boolean expression that is logically equivalent to the truth table presented in this slide.



Truth tables to Boolean expression

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

$A'B'C'$

$A'B'C$

$AB'C$

ABC

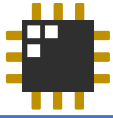
- If either of the several combinations of inputs can be satisfied to make the output 1, that implies the **OR** logic
 - Looking at multiple rows

$$Q = A'B'C' + A'B'C + AB'C + ABC$$

Now, after getting all of those terms, we combine them by ORing the individual terms together.

So now, in English, this basically says that Q will be 1 if and only if, A is 0 and B is 0, and C is 0, OR A = 0 B = 0 C = 1, OR A = 1 B=0 C= 1, and so on. That's exactly the English translation. And thus, this expression exactly describes this truth table.

This is already the Boolean expression that is logically equivalent to the truth table presented in this slide.



Truth tables to Boolean expression

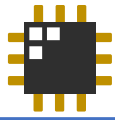
A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- Practice: Write the equivalent Boolean expression of the truth table on the left

Here's a different truth table. Let's try to write the equivalent Boolean expression for this one.

So remember, identify all the rows that has the output equal to 1. Then, for every row, write the equivalent Boolean expression describing that row: ANDing the input conditions together as you go. Then, once you have the expressions per row, OR all of them together to create the final expression.

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$



Truth tables to Boolean expression

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- Practice: Write the equivalent Boolean expression of the truth table on the left

$A'B'C'$

$A'B'C$

$A'BC'$

$A'BC$

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

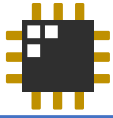
ABC'

ABC

- For the first row, A must be 0, B must be 0, and C must be 0 to make the output Q equal to 1. This is equivalent to $A'B'C'$.
- For the second row, A must be 0, B must be 0, and C must be 1 to make the output Q equal to 1. This is equivalent to $A'B'C$.
- For the third row, A must be 0, B must be 1, and C must be 0 to make the output Q equal to 1. This is equivalent to $A'BC'$.
- For the fourth row, A must be 0, B must be 1, and C must be 1 to make the output Q equal to 1. This is equivalent to $A'BC$.
- For the seventh row, A must be 1, B must be 1, and C must be 0 to make the output Q equal to 1. This is equivalent to ABC' .
- For the eighth row, A must be 1, B must be 1, and C must be 1 to make the output Q equal to 1. This is equivalent to ABC .

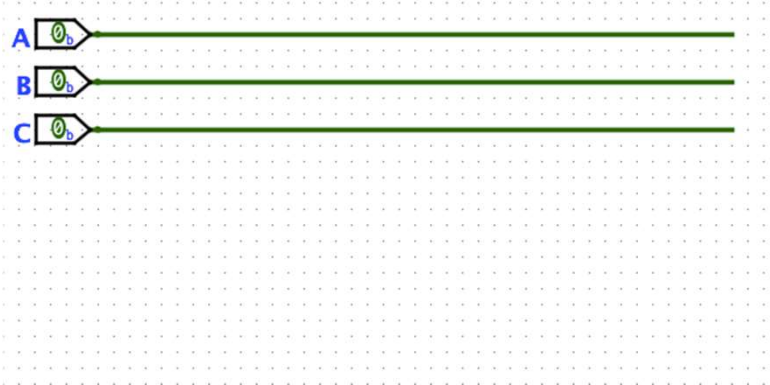
Combining all these terms using the OR operation, we get the Boolean expression

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$



Boolean Expression to Logic Circuit

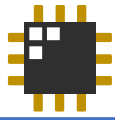
$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$



24

Follow operator order of precedence when analyzing the Boolean expression

- NOT operator evaluated first
 - AND operation
 - OR operation
 - parenthesis change the order of precedence
- Replace each operator with its associated logic gate



Boolean Expression to Logic Circuit

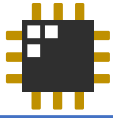
$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$



25

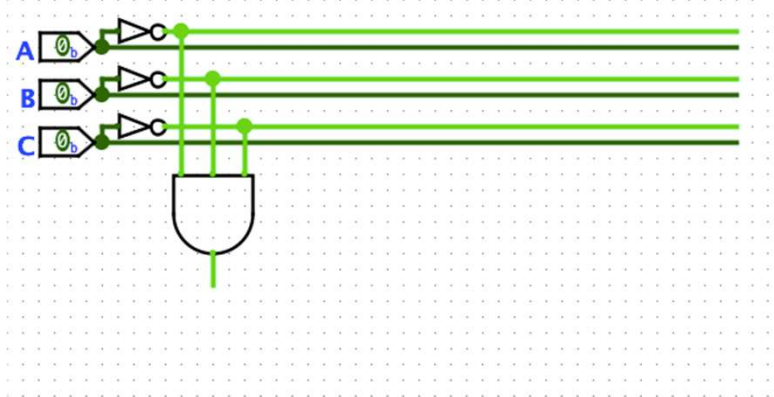
Follow operator order of precedence when analyzing the Boolean expression

- NOT operator evaluated first
 - AND operation
 - OR operation
 - parenthesis change the order of precedence
- Replace each operator with its associated logic gate



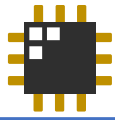
Boolean Expression to Logic Circuit

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$



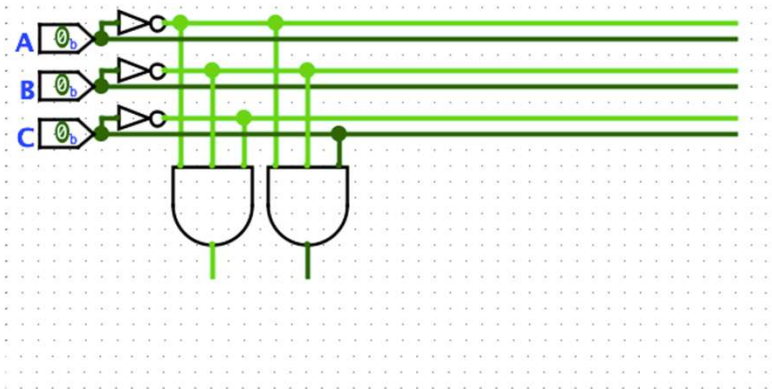
26

Take each term of the expression one at a time and use the appropriate gate to implement it as a logic circuit



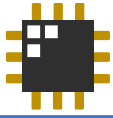
Boolean Expression to Logic Circuit

$$Q = A'B'C' + \boxed{A'B'C} + A'BC' + A'BC + ABC' + ABC$$



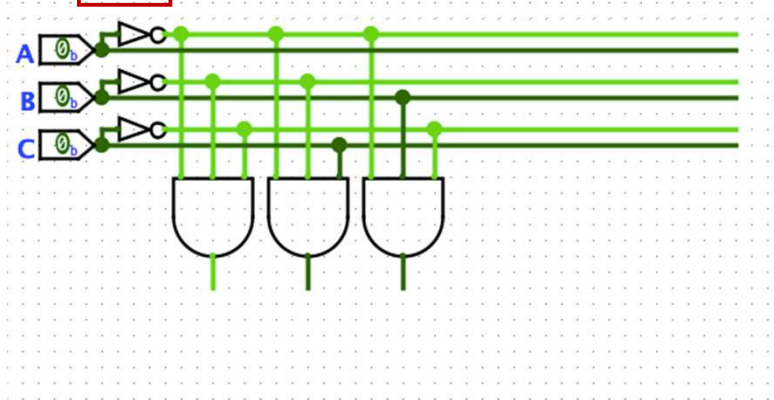
27

Take each term of the expression one at a time and use the appropriate gate to implement it as a logic circuit



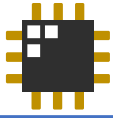
Boolean Expression to Logic Circuit

$$Q = A'B'C' + A'B'C + \boxed{A'BC'} + A'BC + ABC' + ABC$$



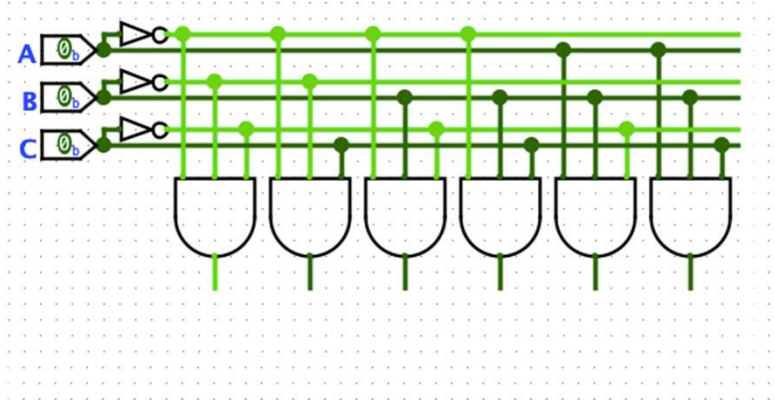
28

Take each term of the expression one at a time and use the appropriate gate to implement it as a logic circuit



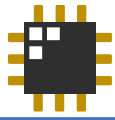
Boolean Expression to Logic Circuit

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$



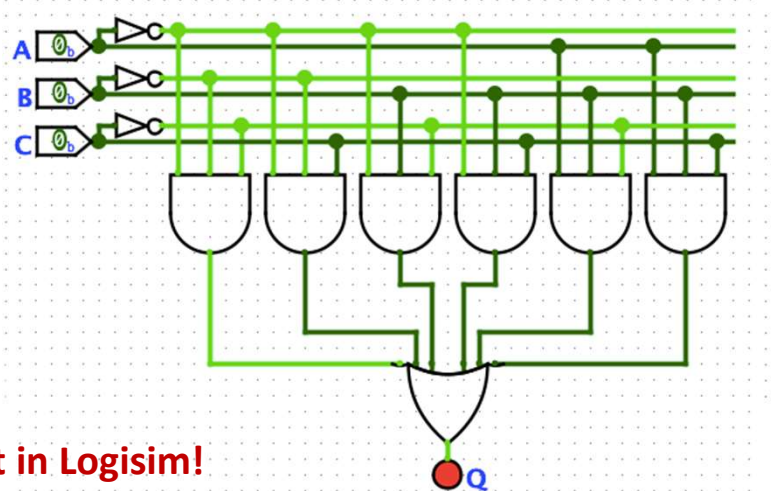
29

Take each term of the expression one at a time and use the appropriate gate to implement it as a logic circuit



Boolean Expression to Logic Circuit

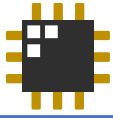
$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$



Try this out in Logisim!

30

For this expression, all the terms are OR'ed together, so we OR all the outputs of our AND gates. Output Q is the output of this OR gate.
Try this out on Logisim.



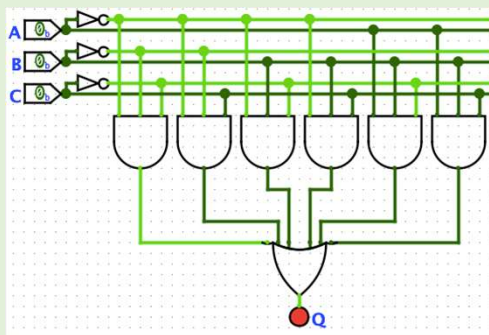
Different representations, same logic behavior

TRUTH TABLE

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

BOOLEAN
EXPRESSION

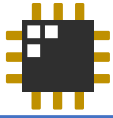


LOGIC
DIAGRAM/
LOGIC CIRCUIT

31

So far, we've gone through a structured process: Starting with a truth table, we derived a Boolean expression. From that Boolean expression, we constructed a logic circuit using logic gates.

Now, looking at this slide, we can see all three representations side by side. While they may appear different visually, they all describe the exact same logic function.



Practice

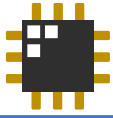
- Implement $F = (AB' + A'C)'$ using any number of AND gates, OR gates, and NOT gates

32

Let's break it down step by step, starting from the innermost expressions:

1. Use a NOT gate to invert B, giving us B' .
2. Use a NOT gate to invert A, giving us A' .
3. Use an AND gate to combine A and B' , giving us AB' .
4. Use an AND gate to combine A' and C, giving us $A'C$.
5. Use an OR gate to combine AB' and $A'C$, giving us $AB' + A'C$.
6. Use a NOT gate to invert the entire expression, giving us $(AB' + A'C)'$.

We used 3 NOT gates, 2 AND gates, and 1 OR gate.



Practice

- Complete the truth table for the expression:

$$Q = A' + B$$

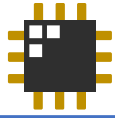
A	B	C	Q
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

33

Given the expression:

$$Q = A' + B$$

We can look at it this way: since **C** is not present in the Boolean expression, it has no effect on the outcome of **Q**. This means we only need to consider **A' + B** when determining **Q**. From the expression, we can see that **Q** will be **1** if **A** is **0** or **B** is **1**; otherwise, **Q** is **0**. The rows where **A** is **0** correspond to the first four rows, while the rows where **B** is **1** are the third, fourth, seventh, and eighth rows. These mentioned rows will have an output of **1**, while all other rows will have an output of **0**.

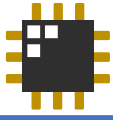


Combinational Circuits

- Combinational circuits are realizations of Boolean functions
- Outputs are a function solely of the inputs at any given time
 - **Memoryless system:** the current state of the outputs only depends on what the inputs are right now
 - Unlike *sequential* circuits (to be discussed in the latter part of 143) where the sequence of inputs or states of the circuit influence what the output will be

34

For any given combination of inputs, there is a corresponding output, which is why a network of logic gates is called a combinational circuit. These circuits are memoryless, meaning their output depends only on the current inputs and not on any previous states. This contrasts with sequential circuits, where the order of inputs and past states influence the output.



Teaser for the next lecture

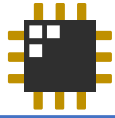
$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

We covered this one
in this lecture

35

Two Boolean expressions can refer to the same truth table! This implies that the two Boolean expressions are logically equivalent.



Teaser for the next lecture

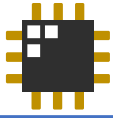
This was the one in the
practice problem earlier

$$Q = A' + B$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

36

Two Boolean expressions can refer to the same truth table! This implies that the two Boolean expressions are logically equivalent.



Teaser for the next lecture

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

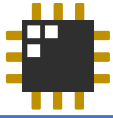
*Tables are the same
Logically equivalent!*

$$Q = A' + B$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

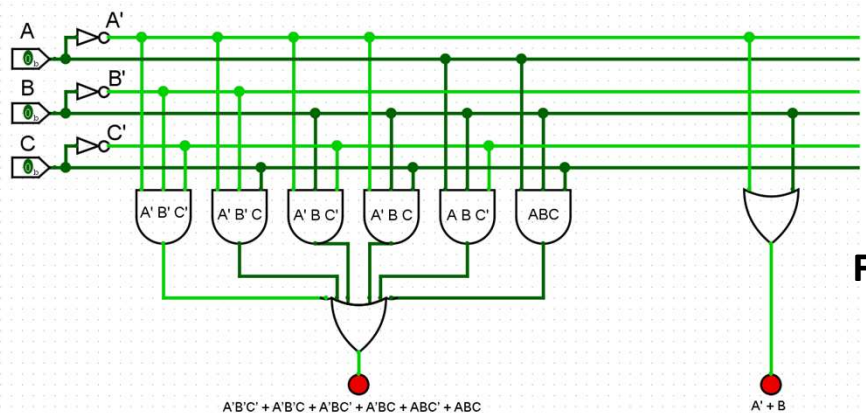
37

Two Boolean expressions can refer to the same truth table! This implies that the two Boolean expressions are logically equivalent.



Teaser for the next lecture

- If the truth tables are the same, they are functionally equivalent!
- Simulation demo:



**Fewer logic gates.
More efficient!**

38

As a final note, we can also perform some simulations to really verify their equivalence. I have a simulator here where you can see the expressions being wired up. You can see the AND gates, OR gates, and NOT gates in there that exactly correspond to how the expressions describe it. Try observing the connections on your own to verify!

On the left is the original expression and its corresponding logic diagram. On the right is the simplified expression and its corresponding diagram. (Simulator: note that when we are changing the inputs to any possible combination, the outputs shown in red behave similarly. This clearly indicates that they behave exactly the same. This further proves that they are logically equivalent.)